

Payment Module Explanation

This document explains how the Payments module works, what data it shows, how it is connected to the database, and what each payment-related table stores.

1. What The Payment Module Is For

The payment module tracks supplier commission payments due to Cash2Switch and agent commission payouts due to employees.

In simple terms:

1. A customer contract is sold or renewed.
2. The system reads the contract, supplier, usage, uplift, term, and agent.
3. The system creates expected supplier commission payment rows.
4. Admin users log supplier receipts when money is received.
5. The system recalculates received/outstanding amounts.
6. Agent commission batches are generated from the supplier receipts.
7. Admin users can mark agent commission batches as paid and download statements.

The module is not a customer billing system. It is a supplier commission and agent payout tracking system.

2. Where The Module Is In The App

Frontend payment pages:

Screen	Frontend file	Purpose
Payment Checker / Upcoming Renewals	cash2switch-frontend/src/app/(main)/dashboard/payments/page.tsx	Main payment schedule list. Shows upcoming commission receipts grouped by contract.
Already Renewed	cash2switch-frontend/src/app/(main)/dashboard/payments/already-renewed/page.tsx	Uses the same Payment Checker page but filters to already renewed contracts.
Supplier Terms	cash2switch-frontend/src/app/(main)/dashboard/payments/supplier-terms/page.tsx	Lets admins configure how each supplier pays commission.
Agent Commissions	cash2switch-frontend/src/app/(main)/dashboard/payments/agent-commissions/page.tsx	Shows receipt-based agent commission batches and payouts.
Reports	cash2switch-frontend/src/app/(main)/dashboard/payments/reports/page.tsx	Management reports for expected, received, outstanding, supplier, agent, and underpaid data.
Customer Payment History	cash2switch-frontend/src/app/(main)/dashboard/payments/history/[client_id]/page.tsx	Shows one customer's payment schedule, receipt history, and agent commission history.

Backend payment API:

Backend file	Purpose
cash2switch-backend/backend/routes/commission_routes.py	Main API routes for supplier terms, payment checker, receipts, reports, and agent commission batches.
cash2switch-backend/backend/utils/commission_schedule.py	Creates commission payment schedules from contract/supplier data.

Backend file	Purpose
cash2switch-backend/backend/utils/commission_backfills.py	Backfills payment schedules for existing already-renewed records.
cash2switch-backend/backend/utils/commission_reminders.py	Moves payment statuses forward and creates CRM notifications for follow-ups.
cash2switch-backend/backend/models.py	SQLAlchemy database models for the payment tables.

3. Database Connection

The backend connects to the database through:

cash2switch-backend/backend/db.py

The connection uses the DATABASE_URL environment variable. If DATABASE_URL is set, the app connects to hosted PostgreSQL using SQLAlchemy and psycopg2. The search path is set to:

```
SET search_path TO "StreemLyne_MT", public
```

That means the payment tables live under the PostgreSQL schema:

```
StreemLyne_MT
```

If DATABASE_URL is missing locally, the backend falls back to SQLite local.db, but production/live usage is PostgreSQL.

4. Main Database Tables

4.1 Supplier_Master

This table stores supplier details and supplier payment rules.

Important payment columns:

Column	Meaning
supplier_id	Supplier primary key.
supplier_company_name	Supplier name, for example BGB, EDF, EON.
commission_payment_type	Main payment policy. Supported values are annual, upfront_reconciliation, monthly_actual, quarterly_actual.
commission_payment_delay_days	Days added to the live/start date or yearly date to calculate due date.
multi_year_commission_payment_mode	Older policy field. Values are annual or upfront. Kept for backwards compatibility.
upfront_percentage	Percentage used when supplier pays upfront and later reconciles.
reconciliation_required	Whether a final reconciliation payment is expected.
invoice_delay_days	Delay for suppliers that pay after invoice/billing.
customer_payment_days	Days allowed for customer payment before supplier payment is expected.
grace_days	Extra buffer days.
commission_payment_frequency	Display/config field: annual, monthly, quarterly, or upfront.

How it is used:

Supplier terms decide how many Commission_Payment rows are created and when they become due.

Example:

? Annual supplier: creates one row per contract year.

? Monthly actual supplier: creates one row per month.

? Quarterly actual supplier: creates one row per quarter.

? Upfront reconciliation supplier: creates an upfront row plus a final reconciliation row.

BGB/BGB Lite/British Gas variants are currently forced to annual/yearly generation in code and supplier config.

4.2 Commission_Payment

This is the main payment schedule table. Each row is one expected supplier commission payment.

Important columns:

Column	Meaning
id	UUID text primary key.
tenant_id	Tenant/company id. Used to isolate records per tenant.
client_id	Links to Client_Master.client_id.
project_id	Links to Project_Details.project_id.
contract_id	Links to Energy_Contract_Master.energy_contract_master_id.
supplier_id	Links to Supplier_Master.supplier_id.
employee_id	Agent/employee attached to the commission.
instalment_year	Sequence number. For annual it is year number. For monthly/quarterly it is the period sequence.
payment_policy_type	The actual policy used when the row was created.
payment_period_label	Human-readable period, for example Year 1, Jan 2026, Quarter 1.
payment_period_start	Start date for the commission period.
payment_period_end	End date for the commission period.
aggregator	Aggregator copied from the contract.
annual_usage	Usage value used for calculation.
uplift	Uplift/rate used for calculation.
contract_term_years	Contract term in years.
live_date	Contract start/live date.
expected_gross_amount	Gross expected commission before Cash2Switch net share.
expected_net_amount	Expected net amount tracked by the system.
due_date	Date payment is expected from supplier.
amount_received	Total received so far against this schedule row.
outstanding_amount	expected_net_amount - amount_received, never below zero.
status	Payment state: Scheduled, Pending, Due, Received, Partially Paid, Chasing Supplier, or Closed.
last_checked_at	Last time status/receipt/follow-up changed.
next_follow_up_date	Next follow-up date if chasing supplier.
follow_up_count	Number of follow-up cycles.
created_at, updated_at	Audit timestamps.

Important rule:

There is a unique constraint on (contract_id, instalment_year). This prevents duplicate payment schedule rows for the same contract sequence.

Old records:

Payment rows with due dates before 2022-01-01 are excluded by the backend filter and are not created by new schedule generation.

4.3 Commission_Payment_Receipt

This table stores actual supplier money received.

One Commission_Payment row can have multiple receipt rows.

Important columns:

Column	Meaning
id	UUID text primary key.
commission_payment_id	Links to Commission_Payment.id.
tenant_id	Tenant id.
amount_received	Amount received from supplier.
date_received	Date the money was received.
notes	Optional notes entered by the user.
logged_by	Employee who logged the receipt.
created_at	When the receipt was created.

How receipt totals work:

When a receipt is added or edited, the backend recalculates the parent Commission_Payment row:

```
amount_received = sum(all receipts for this payment)
outstanding_amount = expected_net_amount - amount_received
```

Status becomes:

- ? Received when outstanding is zero.
- ? Partially Paid when some money is received but outstanding remains.

Receipts can be logged from:

- ? Payment Checker popup.
- ? Customer Payment History page.

Receipts can also be edited from both places.

4.4 Agent_Commission_Batch

This table stores a monthly payout batch for one agent.

Important columns:

Column	Meaning
id	UUID text primary key.
tenant_id	Tenant id.
employee_id	Agent being paid.
batch_month	Month of the payout batch. Stored as first day of month.
total_amount	Total agent commission amount in this batch.
status	Awaiting Payment or Commission Paid.

Column	Meaning
paid_at	When admin marked it paid.
paid_by	Employee/admin who marked it paid.
created_at	When batch was generated.

There is one batch per tenant, employee, and month.

4.5 Agent_Commission_Batch_Item

This table stores the individual receipt lines inside an agent commission batch.

Important columns:

Column	Meaning
id	UUID text primary key.
batch_id	Links to Agent_Commission_Batch.id.
commission_payment_id	Links to the expected payment row.
commission_payment_receipt_id	Links to the supplier receipt.
client_name	Snapshot of customer name at batch generation time.
receipt_amount	Supplier receipt amount used for commission calculation.
commission_rate_snapshot	Agent commission percentage at the time batch item was created.
commission_amount	Calculated amount payable to agent.
created_at	When batch item was created.

Important rule:

There is a unique constraint on `commission_payment_receipt_id`. This prevents the same supplier receipt from being paid to an agent twice.

If a receipt amount is edited later, the backend syncs any existing batch item so the agent commission amount stays aligned with the edited receipt.

4.6 Client_Master

Used by the payment module for:

- ? customer name
- ? tenant id
- ? customer/contact fallback display values

The payment module does not store customer profile data here; it only reads it and stores `client_id` references in payment rows.

4.7 Project_Details

Used by the payment module for:

- ? project id
- ? client id
- ? opportunity id
- ? assigned employee fallback
- ? project status

The Already Renewed page uses project status to separate normal/upcoming renewals from already-renewed contracts.

4.8 Energy_Contract_Master

Used by the payment module for:

- ? contract id
- ? project id
- ? supplier id
- ? employee id
- ? contract start date
- ? contract end date
- ? term sold
- ? MPAN/MPR fields
- ? service id
- ? aggregator
- ? net notch / uplift fallback

This is the key source for creating payment schedules.

4.9 Opportunity_Details

Used as a fallback/source for:

- ? annual usage
- ? uplift
- ? term sold
- ? business/trading name
- ? customer/service context

4.10 Employee_Master

Used for:

- ? agent name
- ? agent id on payment rows
- ? commission percentage

Agent commission calculation uses:

```
agent commission amount = supplier receipt amount * employee commission percentage / 100
```

If an employee's commission_percentage is blank or zero, calculated agent commission will be 0.00.

4.11 Services_Master

Used for service display.

Current frontend/API display rule:

- ? service_id = 1 means Utilities.
- ? service_id = 2 means Water.
- ? Otherwise the service title from Services_Master is shown.

4.12 Client_Interactions

Used by the Already Renewed filter.

The Already Renewed payment page only includes already-renewed contracts where the client interaction notes show:

```
[Renewed by Agent]
```

This prevents customer-renewed contracts from appearing in agent commission payment workflows.

4.13 Notification_Master

Used by the reminder job.

When a commission payment is due or needs follow-up, the backend can create CRM notifications for the assigned employee.

4.14 Dunning_Config

The live database has a table named:

```
StroomLyne_MT.Dunning_Config
```

Columns found in the database:

Column	Meaning
config_id	Primary id for the config row.
plan_id	Optional plan identifier.
retry_schedule	JSONB schedule describing retry/follow-up timing.
max_retries	Maximum number of retries/follow-ups.
grace_period_days	Grace period before retry/follow-up.
is_active	Whether the config is active.
created_at, updated_at	Audit timestamps.

Current status:

- ? The table exists in the live DB.
- ? It currently has no rows.
- ? It is not mapped in backend/models.py.
- ? It is not used by the current payment module routes.

Meaning:

Dunning_Config appears to be a legacy or future configuration table for automated retry/follow-up rules. The current payment module does not depend on it. Current follow-up behavior is handled by `Commission_Payment.status`, `next_follow_up_date`, `follow_up_count`, and `Notification_Master`.

5. How Payment Schedules Are Created

Schedule creation happens in:

```
backend/utils/commission_schedule.py
```

The backend endpoint is:

```
POST /api/commission/generate/<project_id>
```

The function:

```
generate_commission_schedule_for_project(session, project_id)
```

Process:

1. Finds the project in `Project_Details`.
2. Finds the latest contract for that project from `Energy_Contract_Master`.

3. Checks whether Commission_Payment rows already exist for that contract.
4. Reads client data from Client_Master.
5. Reads supplier terms from Supplier_Master.
6. Reads usage/uplift/term from Opportunity_Details, Project_Details, and Energy_Contract_Master.
7. Calculates expected commission.
8. Creates one or more rows in Commission_Payment.

If required data is missing, it does not create rows and returns warnings.

Required data includes:

- ? client
- ? supplier
- ? contract start/live date
- ? annual usage
- ? uplift
- ? contract term
- ? supplier payment policy

6. Commission Calculation

The system calculates expected supplier commission like this:

```
expected gross per year = annual usage * uplift / 100
expected net per year = expected gross per year * 0.80
```

The net amount is what the payment module tracks as expected from the supplier.

For partial contract terms:

```
term fraction = contract term months / 12
expected gross total = expected gross per year * term fraction
expected net total = expected net per year * term fraction
```

Then the total is split into rows depending on supplier policy.

7. Supplier Payment Policies

Annual

Creates one payment row per contract year.

Example for a 3-year contract:

Row	Label	Period
1	Year 1	contract start to end of year 1
2	Year 2	year 2
3	Year 3	year 3

Due date is based on contract start date plus supplier delay days, then repeated yearly.

Upfront Reconciliation

Creates two rows:

1. Upfront payment.

2. Final reconciliation payment.

The upfront row uses `upfront_percentage`.

Example:

If expected net total is GBP 1,000 and upfront percentage is 70%:

? upfront row = GBP 700

? final reconciliation row = GBP 300

Monthly Actual

Creates one row per month.

Due date uses:

```
period_end_date + invoice_delay_days + customer_payment_days + grace_days
```

Quarterly Actual

Creates one row per quarter.

Due date uses the same delay formula as monthly actual.

BGB/BGB Lite/British Gas

The client confirmed BGB is yearly, not monthly.

Current behavior:

? BGB variants are configured as annual/yearly.

? Code also forces BGB variants to annual/yearly as a safety rule.

? Existing BGB monthly records were converted to annual rows.

8. Payment Checker

Frontend:

```
src/app/(main)/dashboard/payments/page.tsx
```

Backend:

```
GET /api/commission/payments
```

Shows payment schedules grouped by contract.

Data displayed includes:

? customer/business name

? contract id

? supplier

? agent

? MPAN/MPR

? contract start date

? contract end date

? service type

? payment period

? expected amount

? received amount

? outstanding amount

? next due date

? status

Filters:

? status

? supplier

? agent

? date range

? search

? already renewed mode

Column selector:

The frontend has a "Show columns" control so users can hide/show optional columns and avoid horizontal congestion.

Click behavior:

Clicking a renewal card/row opens the payment details/history panel.

9. Already Renewed Page

Frontend:

```
src/app/(main)/dashboard/payments/already-renewed/page.tsx
```

This page reuses the Payment Checker implementation with:

```
contract_status=already_renewed
```

Backend filtering:

The backend only returns rows where:

1. Project status is Already Renewed or Renewed Directly.
2. There is a Client_Interactions.notes value containing [Renewed by Agent].
3. Contract start date is within the previous 3 years.

This means:

- ? Renewed by customer records are excluded.
- ? Old already-renewed records outside the 3-year start-date window are excluded.
- ? The page is focused on records where agent commission can apply.

10. Logging Supplier Receipts

Receipts can be logged from:

1. Payment Checker popup.
2. Customer Payment History page.

Backend endpoint:

```
POST /api/commission/payments/<payment_id>/receipts
```

Stored in:

```
Commission_Payment_Receipt
```

Required:

- ? amount received must be greater than zero
- ? date received must be valid, or defaults to current date

After receipt insert:

1. Receipt row is saved.
2. Parent payment totals are recalculated.
3. Parent payment status is updated.

11. Editing Supplier Receipts

Receipts can be edited from:

1. Payment Checker popup.
2. Customer Payment History page.

Backend endpoint:

```
PATCH /api/commission/payments/<payment_id>/receipts/<receipt_id>
```

Editable:

- ? amount received
- ? date received
- ? notes

After receipt edit:

1. Receipt row is updated.
2. If that receipt is already in an agent commission batch, the batch item is recalculated.
3. Parent payment totals are recalculated.

12. Payment Statuses

Allowed statuses:

Status	Meaning
Scheduled	Future payment, not near due date yet.
Pending	Upcoming or expected soon.
Due	Due date has arrived/passed.
Received	Fully received.
Partially Paid	Some received but outstanding remains.
Chasing Supplier	Admin is following up with supplier.
Closed	Manually closed; receipts cannot be added.

Admin users can manually set:

- ? Chasing Supplier
- ? Closed

Backend endpoint:

```
PATCH /api/commission/payments/<payment_id>/status
```

13. Reminder And Follow-Up Logic

Backend file:

```
backend/utils/commission_reminders.py
```

Backend endpoint:

```
POST /api/commission/run-reminders
```

Behavior:

1. Scheduled becomes Pending when due within 30 days.
2. Pending becomes Due when due date is today or earlier.
3. Due/follow-up payments create notifications for assigned employees.
4. Follow-up dates are moved forward after processing.

Current reminder config is stored directly on `Commission_Payment`:

? status

? next_follow_up_date

? follow_up_count

It does not currently use `Dunning_Config`.

14. Agent Commissions

Frontend:

```
src/app/(main)/dashboard/payments/agent-commissions/page.tsx
```

Backend endpoints:

```
GET /api/commission/agent-commissions?month=YYYY-MM
POST /api/commission/batches/generate
POST /api/commission/batches/<batch_id>/mark-paid
GET /api/commission/batches/<batch_id>/statement
```

Agent commission is based on actual supplier receipts, not only expected payments.

Calculation:

```
agent commission = receipt amount * employee commission percentage / 100
```

The employee percentage comes from:

```
Employee_Master.commission_percentage
```

If this value is missing or zero, the agent commission shown will be GBP 0.00.

Batch generation:

1. Admin selects a month.
2. Backend finds supplier receipts received in that month.
3. Backend groups them by employee.
4. Backend creates one `Agent_Commission_Batch` per employee for that month.
5. Backend creates one `Agent_Commission_Batch_Item` per receipt.
6. Backend stores the commission percentage snapshot and calculated amount.

Why snapshots are stored:

If an employee commission percentage changes later, old batch items keep the rate that was used when the batch was generated.

15. Reports

Frontend:

```
src/app/(main)/dashboard/payments/reports/page.tsx
```

Backend endpoints:

```
GET /api/commission/reports/summary
GET /api/commission/reports/by-supplier
GET /api/commission/reports/by-agent
GET /api/commission/reports/underpaid
```

Reports read from:

- ? Commission_Payment
- ? Commission_Payment_Receipt
- ? Supplier_Master
- ? Employee_Master
- ? Client_Master

Reports show:

- ? total expected
- ? total received
- ? total outstanding
- ? supplier totals
- ? agent totals
- ? underpaid/outstanding records

16. Customer Payment History

Frontend:

```
src/app/(main)/dashboard/payments/history/[client_id]/page.tsx
```

Backend:

```
GET /api/commission/customer-log/<client_id>
```

This page shows one customer across:

- ? payment schedule rows
- ? receipt history
- ? agent commission batch items

Admin users can also log and edit supplier receipts here.

17. How Records Are Stored

The module stores three levels of payment data:

Level 1: Expected Payment

Stored in:

```
Commission_Payment
```

This is what the supplier is expected to pay.

Level 2: Actual Supplier Receipt

Stored in:

Commission_Payment_Receipt

This is what the supplier actually paid.

Level 3: Agent Payout

Stored in:

Agent_Commission_Batch
Agent_Commission_Batch_Item

This is what should be paid to the agent based on the supplier receipt.

18. Data Flow Summary

Supplier_Master
defines payment terms

Project_Details + Energy_Contract_Master + Opportunity_Details
provide customer contract, usage, uplift, dates, term, supplier, and agent

generate_commission_schedule_for_project()
creates Commission_Payment rows

Payment Checker
displays Commission_Payment rows

User logs supplier receipt
creates Commission_Payment_Receipt row
recalculates Commission_Payment totals

Generate Month-End Payouts
reads Commission_Payment_Receipt rows
creates Agent_Commission_Batch and Agent_Commission_Batch_Item rows

Admin marks batch paid
updates Agent_Commission_Batch status

19. Common Questions

Why is commission showing as GBP 0.00?

Usually because the agent's Employee_Master.commission_percentage is blank or zero.

The system calculates agent commission from actual supplier receipts:

$$\text{receipt amount} * \text{employee commission percentage} / 100$$

Why are old 2021 records not showing?

The module excludes payment rows with due dates before 2022-01-01. New schedule generation also skips rows before 2022.

Why are Renewed by Customer records not on Already Renewed?

The client requested that only Renewed by Agent records should appear there, because those are the records where agent commission applies.

Why does BGB show yearly?

The client confirmed BGB should be yearly, not monthly. BGB/BGB Lite/British Gas variants are configured and forced as annual/yearly.

What is Dunning_Config?

It is a database table for retry/follow-up configuration, but it is currently empty and not connected to the active payment module code. Current reminders use fields on Commission_Payment and notifications in Notification_Master.

20. Current Important Rules

- ? Payment rows before 2022 are excluded.
- ? BGB/BGB Lite/British Gas payments are yearly.
- ? Already Renewed only includes Renewed by Agent records.
- ? Already Renewed also requires contract start date within the previous 3 years.
- ? Supplier receipt edits recalculate payment totals.
- ? Supplier receipt edits also update agent commission batch items if the receipt was already batched.
- ? Agent commissions are based on actual receipts, not expected payment rows.
- ? If agent commission percentage is zero/missing, the calculated commission is zero.